

Explain Duplicate Skipping

Yes. The key is to understand **three things together**:

1. **Why the first duplicate is not skipped**
2. **Why the second duplicate is skipped at the same recursion level**
3. **Why sorting is absolutely necessary for this condition to work**

Assume:

```
nums = [1, 1, 2]
index 0  1  2
```

and the duplicate condition is:

```
if (i > 0 && nums[i] == nums[i-1] && !used[i-1])
    continue;
```

1. What does this condition actually mean?

Break it into three conditions:

```
i > 0
```

Means:

There must be a previous element.

```
nums[i] == nums[i-1]
```

Means:

Current element is equal to the previous element.

```
!used[i-1]
```

Means:

The previous duplicate is **not currently being used in this permutation branch**.

So the complete meaning is:

"If the current number is a duplicate of the previous number, and the previous duplicate has NOT been used in the current branch, skip the current number."

This prevents the same permutation from being generated twice.

2. Why is the FIRST 1 not skipped?

Initially:

```
nums = [1, 1, 2]
used = [0, 0, 0]
temp = []
```

The loop starts:

```
i = 0
```

Check:

```
if (i > 0 && nums[i] == nums[i-1] && !used[i-1])
```

Substitute:

```
i > 0
0 > 0
```

This is:

```
false
```

And because `&&` needs **all conditions to be true**, the entire condition becomes false.

Therefore:

```
continue;
```

does **not** execute.

So the first `1` is selected.

```
i = 0
↓
nums[0] = 1
↓
i > 0 ? FALSE
↓
ALLOW
```

Important point

The condition cannot skip the first element because:

```
i > 0
```

is false when `i = 0`.

So the first `1` is always allowed.

3. Now why is the SECOND `1` skipped?

After the first `1` has been selected and we backtrack to the root level:

```
temp = []
used = [0, 0, 0]
```

The loop now reaches:

```
i = 1
```

Current element:

```
nums[1] = 1
```

Previous element:

```
nums[0] = 1
```

Now evaluate:

Condition 1

```
i > 0
```

```
1 > 0
```

```
true
```

Condition 2

```
nums[i] == nums[i-1]
```

```
nums[1] == nums[0]
```

```
1 == 1
```

```
true
```

Condition 3

```
!used[i-1]
```

At this point:

```
used = [0, 0, 0]
```

Therefore:

```
!used[0]
!0
true
```

So we have:

```
true && true && true
```

which is:

```
true
```

Therefore:

```
continue;
```

executes.

So:

```
i = 1
↓
nums[1] = 1
↓
same as nums[0]
↓
nums[0] is NOT used
↓
SKIP
```

That's how the second 1 is skipped at the first recursion level.

4. Why don't we skip the second 1 all the time?

This is the most important part.

Suppose we choose the first 1:

```
temp = [1]
used = [1, 0, 0]
```

Now recursion goes deeper.

At this level:

```
i = 1
```

Again:

```
nums[1] == nums[0]
```

is:

```
1 == 1
true
```

But now:

```
used[0] = 1
```

because the first 1 is currently being used.

Therefore:

```
!used[0]
```

becomes:

```
!1
false
```

So:

```
true && true && false
```

becomes:

```
false
```

Therefore `continue` does NOT execute.

The second `1` is allowed.

```
temp = [1a]

    i = 1
    ↓
current = 1b
    ↓
same as previous?
    ↓
YES
    ↓
previous 1 currently used?
    ↓
YES
    ↓
ALLOW
```

So we can create:

```
[1a, 1b]
```

which eventually produces:

```
[1,1,2]
```

5. So what is `!used[i-1]` actually doing?

It distinguishes between **same-level duplicates** and **different-level duplicates**.

Same level

```
[ ]
├─ 1a
├─ 1b ← duplicate branch → SKIP
└─ 2
```

We don't want:

```
1a
```

and

```
1b
```

to create the same set of permutations.

Different recursion levels

```
[ ]
├─ 1a
   └─ 1b
```

Here the second `1` is necessary.

So:

```
!used[i-1]
```

basically says:

```
Skip the duplicate only when its previous copy is NOT being used in the current path.
```

6. Why is SORTING so important?

This is where sorting plays a **major role**.

The condition checks:

```
nums[i] == nums[i-1]
```

It only compares the current element with the **immediately previous element**.

Therefore, duplicates must be next to each other.

For example, after:

```
sort(nums.begin(), nums.end());
```

we get:

```
[1, 1, 2]
```

Now the duplicates are adjacent:

```
  ↑  ↑  
[1, 1, 2]  
 i-1 i
```

So when:

```
i = 1
```

we can detect:

```
nums[1] == nums[0]
```

Without sorting

Imagine:

```
nums = [2, 1, 2]
```

The duplicate 2s are separated:

```
[2, 1, 2]  
 ↑   ↑
```

When we are at the second 2:

```
i = 2
```

the condition checks:

```
nums[2] == nums[1]
```

which is:

```
2 == 1
```

False.

So the duplicate is **not detected**.

That's why sorting is important.

After sorting:

```
[1, 2, 2]
```

Now:

```
nums[2] == nums[1]
```

```
2 == 2
```

and the duplicate can be detected.

7. Complete picture

For:

```
nums = [1a, 1b, 2]
```

after sorting:

```
[1a, 1b, 2]
```

At the first level:

```
temp = []  
  
i = 0 → 1a
```

Allowed because:

```
i > 0  
0 > 0 → false
```

Then backtrack.

```
i = 1 → 1b
```

Check:

```
i > 0           → true  
nums[1] == nums[0] → true  
!used[0]       → true
```

Therefore:

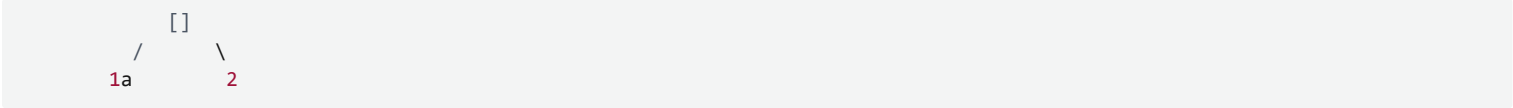
```
true && true && true  
      ↓  
    SKIP
```

Then:

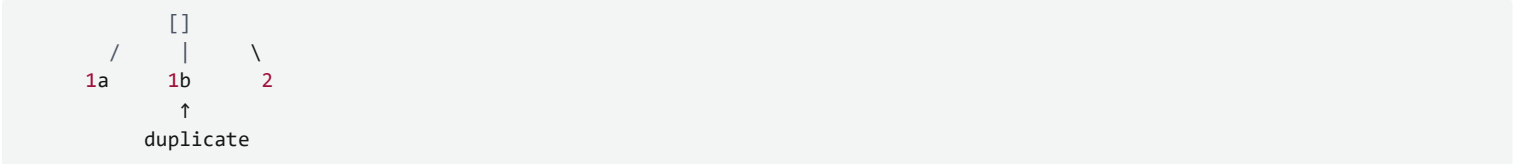
```
i = 2 → 2
```

Allowed.

So first-level choices are:



Not:



8. But after choosing 1a

Now:

```
temp = [1a]  
used = [1,0,0]
```

At the next level:

```
i = 1
```

Check:

```
i > 0           → true  
nums[1] == nums[0] → true  
!used[0]       → !1 = false
```

Therefore:

```
true && true && false  
      ↓  
    false
```

No continue.

So:

```
[1a]
↓
[1a, 1b]
```

is allowed.

The easiest way to remember it

Think of the condition as:

```
if (
  current == previous
  &&
  previous duplicate is NOT in my current path
)
  skip;
```

So:

```
[ ]    → 1a allowed
[ ]    → 1b skipped
```

but:

```
[1a]   → 1b allowed
```

Why?

Because:

previous 1 NOT used → duplicate would create another identical branch → SKIP

previous 1 IS used → we are intentionally using both 1s → ALLOW

And **sorting makes this possible** because it guarantees equal values are adjacent, allowing:

```
nums[i] == nums[i-1]
```

to reliably identify duplicates.